

Frameworks y Usos Estratégicos de Java: Un Análisis del Ecosistema para 2025

Introducción

Java, durante décadas una piedra angular del desarrollo empresarial, afronta 2025 no como una reliquia, sino como un ecosistema dinámico en plena adaptación estratégica. Lejos de la complacencia, la plataforma está experimentando una profunda modernización para abordar sus desafíos históricos y posicionarse como líder en las arquitecturas de *software* de la próxima década.

Este informe analiza el estado de los *frameworks* y los casos de uso de Java para 2025, centrándose en cuatro transformaciones clave que definirán su relevancia:

1. **La Evolución de la Plataforma (Java 25 LTS):** El lanzamiento de un nuevo estándar de Soporte a Largo Plazo (LTS) que reduce las barreras de entrada y solidifica innovaciones críticas.
2. **La Batalla por el Dominio *Cloud-Native*:** La intensa competencia entre el ecosistema dominante de Spring Boot y la nueva generación de *frameworks* optimizados para contenedores (Quarkus, Micronaut).
3. **La Revolución de la Concurrencia (Project Loom):** El impacto de los *Virtual Threads*, que redefine fundamentalmente cómo se escribe y escala el *software* de Java, desafiando al paradigma reactivo.
4. **El Posicionamiento Estratégico en IA y HPC:** La ejecución de una estrategia a largo plazo (Project Babylon y Valhalla) para competir en cargas de trabajo de alto rendimiento, tradicionalmente dominadas por Python y C++.

Este análisis proporciona una evaluación basada en datos sobre la posición del mercado de Java, una comparación técnica de sus *frameworks* y un conjunto de recomendaciones estratégicas para la adopción de tecnología en 2025.

1. Estado del Ecosistema Java en 2025

1.1. Posición en el Mercado: La Doble Cara de la Popularidad

El análisis de la popularidad de Java en 2025 presenta un panorama matizado. El índice TIOBE de octubre de 2025 sitúa a Java en la cuarta posición, con una cuota de *rating* del 8.35%, lo que supone un descenso desde el tercer puesto que ocupaba en octubre de 2024.¹ A pesar de esta ligera caída, el propio índice señala que Java sigue en una "feroz batalla" por el segundo lugar con C y C++, con diferencias de menos del 1%.¹ El mérito clave de Java, según TIOBE, sigue siendo su idoneidad para "grandes aplicaciones empresariales".¹

Sin embargo, los datos del índice TIOBE, que miden el volumen de búsquedas y el "ruido" de la industria (actualmente dominado por el *hype* de la IA que impulsa a Python¹), contrastan con los datos de uso real de la Encuesta de Desarrolladores de Stack Overflow 2025.³ Esta encuesta, que refleja el trabajo real de los desarrolladores, pinta una imagen de salud robusta:

- Java es utilizado por el 29.4% de todos los encuestados y el 29.6% de los desarrolladores profesionales.⁴
- Posee un índice de "Admiración" del 45%.⁴

El indicador adelantado más significativo de la encuesta de Stack Overflow es la demografía de aprendizaje. Un **40.8% de los encuestados que están "Aprendiendo a Programar" utilizan Java**.⁴ Este dato es fundamental, ya que sugiere que el TIOBE, centrado en búsquedas, puede estar capturando una fatiga temporal en el *hype*, mientras que el ecosistema de desarrolladores real está reabasteciendo activamente su reserva de talento. Para las organizaciones, esto mitiga el riesgo de contratación a largo plazo y demuestra que la base académica y de nivel de entrada de Java sigue siendo masiva.

1.2. El Impacto de la Cadencia de Releases: Java 25 como el Nuevo Estándar LTS

Java 25, lanzado en septiembre de 2025⁵, es la nueva versión de Soporte a Largo Plazo (LTS). Esto es un evento crítico para el ecosistema, ya que reemplaza a Java 21 como el estándar *de facto* para la industria empresarial.⁸

Las empresas que priorizan la estabilidad tienden a migrar solo entre versiones LTS. La predecible cadencia de LTS de dos años⁸ permite a las organizaciones adoptar con confianza las innovaciones acumuladas (como los *Virtual Threads* de Java 21) bajo un paraguas de soporte estable.

1.3. Nuevas Capacidades del Lenguaje en Java 25 (JEPs Clave Finalizados)

Java 25 finaliza varias Propuestas de Mejora de JDK (JEPs) que refinan la plataforma, destacando dos por su impacto estratégico:

- JEP 512: Métodos de Instancia `main` y Archivos de Origen Compactos 5
Esta característica elimina la necesidad del tradicional y verboso `public static void main(String args)`.⁵ Un simple `void main() { ... }` es ahora un punto de entrada válido.⁶ Esto no es una característica para arquitectos senior; es una maniobra estratégica de "on-ramping".¹⁰ Reduce drásticamente la "ceremonia" y el "boilerplate" que hacían que Java pareciera intimidante para los principiantes⁵, un punto de fricción clave en la educación en comparación con la simplicidad de Python. Esta JEP, combinada con otras del Proyecto Amber¹⁰, es el paso final de Java para neutralizar la ventaja de Python en la educación y en tareas de scripting sencillas.
- JEP 506: Valores en Ámbito (Scoped Values) (Final) 5
Esta JEP introduce una alternativa ligera, inmutable y segura para hilos a `ThreadLocal`.⁶ Su finalización en un LTS es crucial porque los Scoped Values fueron diseñados específicamente para funcionar con los Virtual Threads (Project Loom).⁶ El patrón `ThreadLocal` es problemático con Loom (debido a la posibilidad de pinning y fugas de memoria). La finalización de Scoped Values solidifica el nuevo modelo de concurrencia de Java (Virtual Threads + Structured Concurrency + Scoped Values) como listo para producción, proporcionando un reemplazo oficial para las arquitecturas basadas en `ThreadLocal`.

Otras características notables finalizadas o en vista previa avanzada incluyen *Module Import Declarations* (JEP 511)⁵, *Flexible Constructor Bodies* (JEP 513)¹⁰ y *Primitive Types in Patterns* (JEP 507).¹⁰

2. La Batalla por el Dominio Cloud-Native: Un Análisis Comparativo de Frameworks

El desarrollo de *backend* en 2025 se define por la tensión entre el ecosistema establecido de Spring y una nueva generación de *frameworks* diseñados explícitamente para arquitecturas *cloud-native* (microservicios, contenedores y *serverless*).

2.1. Spring Boot: El Ecosistema Maduro Enfrenta la Presión del Rendimiento

Spring Boot sigue siendo el "rey"¹² y la "opción por defecto"¹³ para la mayoría de las aplicaciones empresariales. Su fortaleza no es solo el *framework* en sí, sino su vasto ecosistema: un conjunto cohesivo de herramientas como Spring Data, Spring Security, Spring Cloud y Spring Batch⁹ que cubren casi todas las necesidades empresariales. Esto, combinado con una comunidad masiva y un gran *pool* de talento¹⁷, lo convierte en una opción

segura.

Sin embargo, su arquitectura tradicional, basada en la reflexión en tiempo de ejecución (runtime reflection) y la autoconfiguración, lo convierte en un *framework* "pesado".¹⁸ En entornos de contenedores, esto se traduce en altos tiempos de arranque (cold starts) y un consumo de memoria significativo ¹⁸, lo cual es costoso e ineficiente.

2.2. Quarkus: "Supersonic Subatomic Java" para Kubernetes y Serverless

Quarkus es el retador diseñado desde cero para ser "Kubernetes-Native".¹⁵ Su propuesta de valor es clara: tiempos de arranque ultrarrápidos y un uso de memoria drásticamente reducido.¹³

Logra esto al cambiar el trabajo pesado del "tiempo de ejecución" al "tiempo de compilación" ¹⁵ y al adoptar la compilación nativa con GraalVM como un ciudadano de primera clase.¹⁹

Características como *Live Coding* (recarga de código en caliente) ¹⁵ y la compatibilidad con APIs de Spring ¹⁹ están diseñadas para facilitar la migración. Su caso de uso ideal son los microservicios, las funciones *serverless* (AWS Lambda) y cualquier aplicación desplegada en Kubernetes.¹⁵

2.3. Micronaut: Priorizando la Compilación Ahead-of-Time (AOT)

Micronaut comparte los mismos objetivos que Quarkus: un *framework* moderno para microservicios modulares y ligeros.¹³ Su diferenciador filosófico clave es su enfoque en la inyección de dependencias (DI) en *tiempo de compilación*.¹⁵

Micronaut evita la reflexión en tiempo de ejecución casi por completo.¹⁷ Al procesar las anotaciones y generar la configuración en tiempo de compilación (AOT), logra un inicio rápido y un bajo uso de memoria ¹⁹, similar a Quarkus. Es una opción ideal para microservicios y *serverless* donde el mínimo *footprint* de memoria es crítico.¹⁹

2.4. Jakarta EE: El Estándar Empresarial Confiable

Jakarta EE, la evolución de Java EE bajo la Eclipse Foundation ¹⁵, sigue siendo el "caballo de batalla empresarial".¹² Su fortaleza radica en la estabilidad, madurez, cumplimiento y estandarización. Es la opción preferida en sectores altamente regulados y conservadores como la banca, los seguros y el gobierno.¹⁵ Sin embargo, su naturaleza más pesada y su diseño orientado a servidores de aplicaciones la hacen menos ágil para casos de uso reactivos o *serverless* ¹⁵, donde Spring Boot o Quarkus ofrecen más agilidad.

2.5. La Verdadera Batalla: Tiempo-de-Compilación vs. Tiempo-de-Ejecución

La división entre estos *frameworks* es una división filosófica de arquitectura. Spring Boot ¹⁷ tradicionalmente realiza su "magia" (DI, autoconfiguración) en *tiempo de ejecución*. Esto ofrece una gran flexibilidad dinámica, pero a costa de un rendimiento de inicio lento y un alto consumo de memoria.¹⁸

Quarkus ¹⁵ y Micronaut ²⁰ mueven este trabajo al *tiempo de compilación*. "Hornean" la configuración en el artefacto final, resultando en un ejecutable nativo optimizado y rápido.¹⁹ En el mundo *cloud-native* de 2025, donde los contenedores se crean y destruyen constantemente (scale-to-zero), el rendimiento del *arranque* se ha vuelto más crítico que la flexibilidad del *runtime*.

Esto no significa que Spring esté obsoleto. Spring es más que un *framework*; es una plataforma de ecosistema (Data, Security, Cloud).¹⁵ El costo de migrar *fuera* de este ecosistema cohesivo es astronómicamente alto. Quarkus lo entiende, y su capa de compatibilidad con APIs de Spring ¹⁹ es una "rampa de migración" inteligente.

La elección estratégica para 2025 es clara:

- **Proyectos Nuevos (Greenfield):** Para microservicios críticos en rendimiento o *serverless*, Quarkus y Micronaut son la opción técnica superior.¹³
- **Sistemas Existentes (Brownfield):** Spring sigue siendo el centro de gravedad. El movimiento defensivo de Spring (Spring Boot 3+ con soporte AOT/GraalVM ²³) busca ser "suficientemente rápido" para detener la migración de sus usuarios.

3. La Revolución del Rendimiento: GraalVM, Compilación Nativa (AOT) y el Futuro de la JVM

La tecnología habilitadora detrás de la batalla *cloud-native* es GraalVM y la compilación Ahead-of-Time (AOT).

3.1. Análisis de Métricas: JVM vs. Imagen Nativa

El compilador Just-in-Time (JIT) de la JVM tradicional es increíblemente potente para optimizar el rendimiento de aplicaciones de larga duración, pero sufre de "retrasos de inicio" ²⁵ y una sobrecarga de memoria ²⁵ mientras calienta. Esto es fatal para las funciones *serverless* y los microservicios que necesitan escalar rápidamente.²⁴

La solución es GraalVM Native Image.²⁴ Esta tecnología compila el código Java AOT en un

ejecutable nativo específico de la plataforma, eliminando el JIT en tiempo de ejecución. Los beneficios estratégicos son claros ²⁸:

- **Inicio Rápido (Fast Startup):** Se logran tiempos de inicio en milisegundos (sub-segundo).²⁴
- **Bajo Uso de Memoria (Low Resource Usage):** Requiere significativamente menos CPU y RAM.²⁸
- **Empaquetado Compacto (Compact Packaging):** Produce imágenes de contenedor más pequeñas y ligeras.²⁸

Quarkus ¹⁵ y Micronaut ¹⁵ fueron diseñados para esto. Spring Boot 3+ ahora también soporta AOT ²³ como respuesta a esta tendencia.

3.2. Tabla 1: Comparativa de Métricas de Rendimiento de Frameworks Cloud-Native (2025)

Los datos de *benchmarks* de 2025 cuantifican el *trade-off* entre los *frameworks*. La siguiente tabla sintetiza datos de *benchmarks* públicos ³⁰ para una aplicación REST simple, comparando el modo JVM (JIT) con el modo Nativo (AOT).

Framework (Versión 2025)	Modo	Tiempo de Arranque (ms)	Memoria RSS Máx. (MB)	Rendimiento Sostenido (Peticiones/seg)
Spring Boot 3.3+	JVM (JIT)	~1,909 ms ³⁰	~388.9 MB ³⁰	Alto (tras calentamiento) ³²
Spring Boot 3.3+	Nativo (AOT)	~104 ms ³⁰	~149.4 MB ³⁰	Varía (potencialmente menor que JIT)
Quarkus 3	JVM (JIT)	~1,154 ms ³⁰	~271.2 MB ³⁰	Competitivo
Quarkus 3	Nativo (AOT)	~49 ms ³⁰	~70.5 MB ³⁰	Alto ³¹
Micronaut 4	JVM (JIT)	~656 ms ³⁰	~253.2 MB ³⁰	Competitivo
Micronaut 4	Nativo (AOT)	~50 ms ³⁰	~83.8 MB ³⁰	Alto

Nota: Las métricas de rendimiento (Arranque, Memoria) se basan en un benchmark de 2025 ³⁰ para una aplicación web simple con JDBC, Flyway y Postgres. El rendimiento sostenido se basa en análisis cualitativos.³¹

3.3. El Costo Oculto de la AOT: El "Pacto Faustiano" del "Mundo Cerrado"

La adopción de AOT no es gratuita. Tiene un "costo enorme" ³⁴ y "no es 'plug and play'".³⁴ La

compilación AOT se basa en una "suposición de mundo cerrado" (Closed-World assumption)³⁴: debe conocer *todo* el código alcanzable en *tiempo de compilación*.²⁷

Esto choca frontalmente con el poder histórico de Java y Spring, que se basaba en un "mundo abierto" (reflexión, carga dinámica de clases, *proxies*). Cuando el compilador AOT no puede ver código dinámico, falla. El síntoma es que los desarrolladores deben gastar "tiempo infinito"³⁴ escribiendo configuraciones manuales (*reflect-config*)³⁴ para informarle al compilador sobre este comportamiento dinámico.

Aquí radica el verdadero valor de Quarkus y Micronaut: no es solo *usar* GraalVM, sino *gestionar* su complejidad. Sus arquitecturas AOT-first¹⁵ están diseñadas para *eliminar* la necesidad de esta configuración manual. Por el contrario, adaptar una aplicación Spring legacy compleja a Spring 3+ AOT²³ será significativamente más difícil, porque el ADN de Spring es fundamentalmente dinámico.¹⁷

Además, el JIT no está obsoleto. Un *benchmark*³² reveló que, si bien Quarkus gana el arranque, "bajo carga pesada, la optimización JIT de Spring Boot se rió en la cara de Quarkus". El JIT recopila datos de *profiling en vivo*²⁵ y realiza optimizaciones dinámicas que el AOT (estático) no puede prever.

La elección de arquitectura depende del caso de uso:

- **Serverless / Scale-to-Zero:** El arranque (cold start) lo es todo. AOT/Nativo es la única opción.²⁴
- **Monolito / Servicio de Larga Duración:** El arranque es irrelevante. El rendimiento pico sostenido³² lo es todo. El JIT (JVM) tradicional sigue siendo una opción superior.

4. El Nuevo Paradigma de Concurrency: Project Loom vs. Programación Reactiva

El cambio arquitectónico más significativo en Java en 2025 es la concurrencia, gracias a la finalización de Project Loom.

4.1. Project Loom (Virtual Threads): Simplificando la Concurrency

- **El Problema:** Los hilos de plataforma (hilos del SO) son "pesados" y "caros".³⁵ Cada uno consume ~1MB de memoria de pila.³⁵ Escalar más allá de unos pocos miles de hilos es imposible, y las operaciones de E/S (I/O) bloqueantes (consultas a DB, llamadas a API) desperdician estos costosos recursos.³⁶
- **La Solución (Loom):** Los *Virtual Threads*²⁴ son hilos "ligeros" y "baratos" gestionados por la JVM, no por el SO.³⁶ Millones de hilos virtuales pueden ejecutarse en un puñado de hilos del SO.³⁵
- **El Beneficio:** Los desarrolladores pueden volver a escribir código *bloqueante*

(imperativo, síncrono) simple y legible.³⁵ Cuando un hilo virtual bloquea (p.ej., esperando una respuesta de la DB), la JVM lo "desmonta" de su hilo portador del SO y permite que ese hilo del SO haga otro trabajo.³⁵ Esto proporciona escalabilidad asíncrona con un modelo de programación síncrono, lo que facilita enormemente la escritura, la lectura y la depuración del código.³⁶

4.2. El Ecosistema Reactivo (Project Reactor y RxJava)

Antes de Loom, la única forma de lograr una alta concurrencia era con la programación asíncrona no bloqueante, lo que dio lugar a los *frameworks reactivos*.³⁶ Este paradigma, basado en flujos de datos (data streams)³⁹, está dominado por Project Reactor (base de Spring WebFlux)⁴⁰ y el pionero RxJava.⁴⁰

Si bien son potentes, estos *frameworks* son famosos por su alta complejidad. Son difíciles de escribir, leer y depurar (a menudo resultando en "callback hell").³⁶ Brian Goetz, arquitecto de Java en Oracle, ha descrito la programación reactiva como una "tecnología de transición"³⁶ cuya principal motivación fue eludida por Loom.

4.3. Debate 2025: ¿Reemplazarán los Virtual Threads al Modelo Reactivo?

La opinión predominante es que, para la mayoría de las aplicaciones de negocio (APIs REST, CRUD), Project Loom "matará" la *necesidad* de la programación reactiva.³⁶ El código es "vasta-mente más simple"³⁶ y la principal razón para usar Reactivo (escalar la E/S) ha sido eliminada por Loom.⁴¹

Sin embargo, este debate es una falsa dicotomía. Es una confusión entre "concurrencia" y "procesamiento de flujos" (streaming).

La programación reactiva tiene una característica única e irremplazable que Loom no aborda: Backpressure (contra-presión).³⁶

Backpressure es el mecanismo que permite a un consumidor de datos lento (p.ej., una base de datos) *indicar* a un productor de datos rápido (p.ej., un *stream* de Kafka) que reduzca la velocidad, evitando así ser sobrecargado.³⁶ Los *Virtual Threads* no solo no tienen este mecanismo, sino que *empeoran* el problema: ahora es trivialmente fácil para un productor generar millones de hilos virtuales y *sobrecargar* a un consumidor.³⁶

Esto conduce a la recomendación estratégica de 2025:

- **Caso de Uso 1 (API Concurrente):** Múltiples solicitudes (p.ej., API REST, microservicios) que realizan E/S bloqueante.
 - **Ganador: Virtual Threads (Loom).**
- **Caso de Uso 2 (Procesamiento de Flujos):** Consumir *streams* de datos ilimitados (p.ej., un tópico de Kafka, pipelines de datos en tiempo real).

- **Ganador: Programación Reactiva (Project Reactor).**

Project Loom *sana* la división del ecosistema Spring. Durante años, los equipos se vieron *forzados* ³⁵ a usar Spring WebFlux (Reactivo) ⁴¹, no porque quisieran el paradigma, sino porque necesitaban la escalabilidad no bloqueante. Ahora, Spring MVC (con Loom) se convierte en la opción superior para el Caso de Uso 1. Spring WebFlux vuelve a ser una herramienta de nicho, pero indispensable, para el Caso de Uso 2.

4.4. Adopción en Frameworks y el Próximo Cuello de Botella

La adopción de Loom varía en simplicidad:

- **Spring Boot:** Trivial. Se habilita con una sola línea de configuración:
`spring.threads.virtual.enabled: true.`³⁵
- **Quarkus:** Integración explícita mediante la anotación `@RunOnVirtualThread.`⁴³
- **Micronaut:** Introduce un "modo portador de loom" experimental ⁴⁵ para Netty.

Sin embargo, habilitar Loom no es una solución mágica. Un *benchmark* de 2025 que probó Spring Boot + Loom + PostgreSQL encontró que el rendimiento *no* mejoró como se esperaba.⁴⁷ El cuello de botella simplemente se movió. Los *pools* de conexiones de bases de datos (como HikariCP) ⁴⁷ fueron diseñados para un mundo de *pocos hilos caros*. Usan bloques `synchronized` ⁴⁷ para gestionar un pequeño *pool* de conexiones. Cuando millones de hilos virtuales intentan obtener una conexión, todos se *bloquean* en ese único punto de sincronización. Este bloqueo "fija" (pins) el hilo virtual a su hilo portador del SO ³⁶, anulando todo el beneficio de Loom.

La implicación estratégica es clara: habilitar Loom es el paso 1. El paso 2 es rediseñar el ecosistema, reemplazando `ThreadLocal` con `ScopedValue` ⁶ y reemplazando los *pools* de conexiones por unos "conscientes de Loom".⁴⁷

5. Java como Plataforma Estratégica para Inteligencia Artificial y Machine Learning

Históricamente, Python ha eclipsado a Java en IA. Sin embargo, en 2025, Java está ejecutando una estrategia de nicho para la inferencia empresarial y sentando las bases (Babylon, Valhalla) para competir en rendimiento.

5.1. El Doble Rol de Java: Inferencia Empresarial vs. Entrenamiento (Dominio de Python)

El *statu quo* de la industria de IA tiene una clara división:

- **Dominio de Python:** Python domina la *investigación*, el *prototipado* y el *entrenamiento* de modelos.⁴⁸ Su sintaxis simple⁴⁸ y su ecosistema de bibliotecas (TensorFlow, PyTorch, scikit-learn)⁴⁸ son imbatibles para los científicos de datos.
- **Nicho de Java (Inferencia):** Java sobresale en la *implementación* (deployment) e *inferencia* en producción.⁴⁸ Las empresas no ejecutan sus modelos en *notebooks* de Python; los implementan en microservicios robustos, seguros y de baja latencia. Aquí es donde la estabilidad de la JVM, el rendimiento y la integración empresarial de Java⁴⁸ son superiores.

Las bibliotecas de Java ML (como DeepLearning4j⁵⁰ y Tribuo⁵²) existen, pero su característica más importante es la capacidad de *importar* y *ejecutar* modelos entrenados en Python (p.ej., Tribuo puede desplegar modelos de scikit-learn y PyTorch).⁵²

Este "problema de los dos lenguajes" (entrenar en Python, desplegar en Java) es una fuente importante de fricción, riesgo y latencia en la IA empresarial.⁴⁸ El objetivo estratégico de Java no es reemplazar a Python en el *entrenamiento*, sino *eliminar el problema de los dos lenguajes* al convertirse en la mejor plataforma de *inferencia*.

5.2. Proyectos Estratégicos de OpenJDK para la Dominancia en IA

OpenJDK está ejecutando una estrategia de pinza con dos proyectos futuros clave:

5.2.1. Project Babylon: Acceso Directo a GPUs y Runtimes de IA

54

El objetivo de Babylon es extender Java a "modelos de programación extranjeros" como la IA y las GPUs.⁵⁵ Utiliza dos componentes clave:

1. **API FFM (Foreign Function & Memory):** Permite a Java llamar a código nativo (C) de forma segura y eficiente.⁵⁴
2. **Runtimes Nativos:** Babylon usa FFM para conectarse a *runtimes* de IA como **ONNX**.⁵⁴ Esto permite que un modelo entrenado en cualquier *framework* (PyTorch, TF) y exportado al formato ONNX sea ejecutado por Java con **aceleración de GPU**.⁵⁴

Un subproyecto, **HAT (Heterogeneous Accelerator Toolkit)**⁵⁴, incluso permite a los desarrolladores escribir *kernels* de cómputo (similares a CUDA) en Java puro para ser ejecutados en la GPU.⁵⁴

5.2.2. Project Valhalla: Optimizando la Disposición de Memoria para Cargas de Trabajo de IA

57

El objetivo de Valhalla es introducir "Tipos de Valor" (Value Types) o "Clases Primitivas".⁶¹ Estos son "objetos sin identidad" ⁶¹, que combinan la eficiencia de los structs de C con la seguridad de Java.⁶¹

Esto es fundamental para la IA, que es esencialmente matemática de matrices.⁶¹

- **Antes de Valhalla:** Un Array era un *array de punteros* a objetos Vector3D dispersos por el *heap*.⁶¹ Esto es un desastre para la caché del CPU.⁶¹
- **Después de Valhalla:** Un Array será un *único bloque de memoria contiguo* [x1, y1, z1, x2, y2, z2,...].⁶³ Este "aplanamiento" (flattening) de la memoria ⁶¹ es crucial para el cómputo de alto rendimiento.

5.3. La Convergencia "Babylon + Valhalla": La Estrategia de Pinza de Java para la IA

Estos dos proyectos no son independientes; están profundamente conectados. Para ejecutar algo en una GPU (Babylon), primero se deben *transferir los datos* desde la memoria de la CPU a la memoria de la GPU. Si esos datos son un conjunto de punteros dispersos (Java tradicional), esta transferencia es extremadamente lenta. Pero si los datos son un *único bloque de memoria contiguo* (Valhalla), la transferencia (a través de la API FFM) es casi instantánea.

Valhalla proporciona las *estructuras de datos* eficientes ⁶³ y Babylon proporciona el *pipeline de cómputo*.⁵⁴ Juntos, representan la visión a largo plazo (2026-2028) de Java para competir con C++ y Python no solo en integración, sino en *rendimiento puro* de IA.⁵⁷

6. Dominios de Aplicación Especializados

6.1. Big Data: Java como Lenguaje de Procesamiento de Flujo en Tiempo Real

Java sigue siendo un lenguaje central para procesar grandes volúmenes de datos ⁶⁷, siendo la base de los motores de procesamiento más importantes del mundo.

- **Apache Spark (Structured Streaming):** Originalmente un motor de *batch*, ahora procesa *streaming* mediante "micro-lotes".⁶⁹ Su fortaleza es la API unificada ⁷⁰ y la profunda integración con el ecosistema *lakehouse* (Delta, Hudi, Iceberg).⁷⁰ Soporta Java.⁶⁹
- **Apache Flink:** Un motor *streaming-first*.⁶⁹ Procesa *récord por récord* ⁷⁰, ofreciendo una latencia ultra baja (milisegundos).⁶⁹ Es superior en la gestión de estado.⁶⁹ Soporta

Java.⁶⁹

- **Apache Kafka Streams:** Es una *biblioteca* de Java/Scala, no un *framework* de clúster.⁷⁰ Es ligera e ideal para construir microservicios de *streaming* que leen y escriben directamente en Kafka.⁷⁰

No hay un "mejor" motor. La elección es un *trade-off*: Flink es el *especialista* para latencia ultra baja; Spark es el *generalista* para unificación de *batch/streaming* y *lakehouse*; y Kafka Streams es la solución *ligera* para microservicios Kafka-nativos.⁷⁰

6.2. Tabla 2: Análisis Comparativo de Motores de Streaming de Big Data (2025)

La siguiente tabla, basada en un análisis comparativo detallado ⁷⁰, resume los casos de uso ideales para los motores de *streaming* basados en JVM.

Motor	Modelo de Procesamiento	Latencia	Gestión de Estado	Caso de Uso Ideal
Apache Flink	Streaming Nativo (Récord-por-Récord)	Ultra Baja (ms)	Superior (Nativa)	"Mejor en su clase" para transacciones de ultra-baja latencia.
Spark Structured Streaming	Micro-Lotes	Baja (segundos)	Buena (Checkpointing)	"Muy adecuado" para unificación de <i>stream/batch</i> e integración <i>lakehouse</i> .
Apache Kafka Streams	Streaming Nativo (Récord-por-Récord)	Baja (segundos)	Buena (Local/RocksDB)	"Mejor en su clase" para microservicios ligeros <i>event-driven</i> (Kafka-nativo).

6.3. Desarrollo Móvil: El Rol de Java frente a Kotlin en el Ecosistema Android 2025

En el dominio móvil, la batalla ha terminado. Google declaró oficialmente a **Kotlin como el lenguaje "preferido"** para el desarrollo de Android.⁷¹

Kotlin ganó por ofrecer una sintaxis más limpia, menos *boilerplate* ⁷¹ y, de forma crucial, una

mejor **seguridad contra nulos (null safety)** ⁷¹, que reduce directamente la cantidad de *crashes* de las aplicaciones.⁷¹

El rol de Java en Android para 2025 es el de lenguaje *legacy* (heredado).⁷¹ Aún es necesario para mantener la vasta base de código existente ⁷¹, pero no debe usarse para iniciar nuevos proyectos. El desarrollo moderno de Android se centra en **Jetpack Compose** (para UI) ⁷⁴ y **Kotlin Multiplatform (KMP)** ⁷⁵, ambos ecosistemas centrados en Kotlin. Para el desarrollo de Android, Java está en modo de mantenimiento.

6.4. Desarrollo de UI: El Nicho de los Frameworks Full-Stack y de Escritorio

- **JavaFX:** Es el sucesor de Swing y la opción moderna para construir aplicaciones de *escritorio* (desktop) robustas en Java.⁷⁶
- **Vaadin:** Este es un *framework full-stack* para construir aplicaciones *web* interactivas usando **100% Java**.¹² Vaadin abstrae las complejidades del *frontend* (JS, HTML, CSS) ⁷⁶, permitiendo que la lógica de la UI se ejecute en el *servidor*.⁷⁶

El nicho de Vaadin es el de la pila empresarial "Anti-JavaScript". Muchas organizaciones empresariales tienen equipos de desarrolladores *backend* de Java ¹² que no quieren, o no pueden, gestionar la complejidad de un *stack* de JavaScript moderno (NPM, Webpack, React, etc.). Vaadin ⁷⁶ les ofrece un camino de baja fricción para construir aplicaciones *internas* (paneles de control, *dashboards* ¹²) de forma rápida y segura ⁷⁹ usando solo el equipo y las habilidades que ya poseen.

7. Conclusión: Recomendaciones Estratégicas para la Adopción de Java en 2025

7.1. Síntesis de la Tesis: Java en 2025 - Adaptación y Dominio Continuo

El ecosistema de Java en 2025 es vibrante y está definido por una adaptación estratégica exitosa. Ha neutralizado la amenaza de la simplicidad de Python en la educación (JEP 512) ⁶, está en proceso de neutralizar la amenaza del rendimiento de los contenedores (Quarkus, Micronaut, AOT) ²⁴ y ha resuelto su crisis de concurrencia (Loom).³⁵ Simultáneamente, está sentando las bases (Babylon, Valhalla) ⁵⁵ para su próxima batalla en la IA de alto rendimiento.

7.2. Recomendaciones de Pila Tecnológica (Tech Stack) para 2025

Basado en este análisis, se prescriben las siguientes recomendaciones para los líderes técnicos:

- **Para Nuevos Proyectos (Greenfield) Cloud-Native / Serverless:**
 - **Elección:** Quarkus¹⁹ o Micronaut.¹⁹
 - **Justificación:** Rendimiento superior (arranque, memoria) (Tabla 1)³⁰, optimizado para GraalVM AOT¹⁵ y listo para Kubernetes.¹⁸
- **Para Sistemas Empresariales Existentes (Brownfield) / Ecosistemas Complejos:**
 - **Elección:** Spring Boot 3+.¹⁴
 - **Justificación:** Madurez del ecosistema¹⁶, disponibilidad de talento¹⁷ y la nueva escalabilidad de **Virtual Threads (Loom)**.³⁵ La actualización a Java 21+ (LTS) y la habilitación de Loom es la actualización de mayor ROI.
- **Para Aplicaciones de Alto Rendimiento y Baja Latencia (API):**
 - **Elección:** Spring MVC con Virtual Threads³⁵ o Quarkus con @RunOnVirtualThread.⁴⁴
 - **Evitar:** Programación Reactiva (WebFlux), a menos que el *backpressure* sea un requisito (Sección 4.3).
- **Para Pipelines de Datos en Tiempo Real (Streaming):**
 - **Elección:** Project Reactor (Reactivo)⁴⁰ si el *backpressure* es necesario.³⁶
 - **Elección de Motor:** Apache Flink⁷⁰ para latencia de milisegundos; Apache Spark⁷⁰ para integración general de *lakehouse* (Tabla 2).
- **Para Estrategias de IA / ML:**
 - **Elección:** Implementar modelos (entrenados en Python) usando ONNX⁵⁴ y ejecutarlos a través de bibliotecas Java como Tribuo⁵² o la API FFM⁵⁸ para inferencia de baja latencia.
- **Para Desarrollo Móvil (Android) y UI:**
 - **Android:** Kotlin.⁷¹ Java es solo para mantenimiento.⁷¹
 - **UI Web Interna:** Vaadin⁷⁶ para equipos de Java puros.
 - **UI de Escritorio:** JavaFX.⁷⁶

7.3. Visión a Futuro: El Horizonte 2026-2028

Los líderes técnicos deben monitorear de cerca el progreso de **Project Valhalla**⁶¹ y **Project Babylon**.⁵⁵ La convergencia de estos proyectos (Sección 5.3) representa el siguiente gran salto de rendimiento para Java, con el objetivo de unificar el desarrollo de aplicaciones empresariales y el cómputo de IA de alto rendimiento en una sola plataforma.

Fuentes citadas

1. TIOBE Index - TIOBE - TIOBE Software, acceso: noviembre 9, 2025,

- <https://www.tiobe.com/tiobe-index/>
2. Top Programming Languages to Learn in 2025: What the Data Says - Lemon.io, acceso: noviembre 9, 2025, <https://lemon.io/blog/most-popular-programming-languages/>
 3. 2025 Stack Overflow Developer Survey, acceso: noviembre 9, 2025, <https://survey.stackoverflow.co/2025/>
 4. Technology | 2025 Stack Overflow Developer Survey, acceso: noviembre 9, 2025, <https://survey.stackoverflow.co/2025/technology>
 5. Java 25: The Future of Coding Made Easy | by Java Techie | Oct, 2025, acceso: noviembre 9, 2025, <https://medium.com/@javatechie/java-25-the-future-of-coding-made-easy-de559a65df2f>
 6. New Features in Java 25 | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-25-features>
 7. Java 25 New Features With Examples : r/programming - Reddit, acceso: noviembre 9, 2025, https://www.reddit.com/r/programming/comments/1nqupsm/java_25_new_features_with_examples/
 8. 10 Emerging Java Trends to Watch Out for in 2025 - Blog | Miracle Software Systems, acceso: noviembre 9, 2025, <https://blog.miraclesoft.com/10-emerging-java-trends-to-watch-out-for-in-2025/>
 9. 🔥 Java 25 | Game Changer! 🚀 Top Features Explained with Code @Javatechie, acceso: noviembre 9, 2025, <https://www.youtube.com/watch?v=xvHhO0SN2xc>
 10. All New Java Language Features Since Java 21 #RoadTo25 - YouTube, acceso: noviembre 9, 2025, <https://www.youtube.com/watch?v=X0-TGhktFnE>
 11. JDK 25 Release Notes, Important Changes, and Information - Oracle, acceso: noviembre 9, 2025, <https://www.oracle.com/java/technologies/javase/25-relnote-issues.html>
 12. Best Java Frameworks 2025: The Tools Powering the Next Generation of Web and Backend Development! - Enstacked, acceso: noviembre 9, 2025, <https://medium.enstacked.com/best-java-frameworks-2025-the-tools-powering-the-next-generation-of-web-and-backend-development-96869ce4bf23>
 13. Choosing the Right Java Microservices Framework: Spring Boot, Quarkus, Micronaut, and Beyond - Igor Venturelli, acceso: noviembre 9, 2025, <https://igventurelli.io/choosing-the-right-java-microservices-framework-spring-boot-quarkus-micronaut-and-beyond/>
 14. Most Popular Java Web Frameworks in 2025 - Rollbar, acceso: noviembre 9, 2025, <https://rollbar.com/blog/most-popular-java-web-frameworks/>
 15. Top Frameworks for Java Cloud Development - Brilworks, acceso: noviembre 9, 2025, <https://www.brilworks.com/blog/frameworks-for-java-cloud-development/>
 16. Java Frameworks Guide 2025: Best Options for Business Development - Softjournal, acceso: noviembre 9, 2025, <https://softjournal.com/insights/java-frameworks>
 17. Spring Boot vs Quarkus vs Micronaut: Which Java Framework Should You

- Choose? | by Brilworks Software | Sep, 2025 | Medium, acceso: noviembre 9, 2025,
<https://medium.com/@Brilworks/spring-boot-vs-quarkus-vs-micronaut-performance-and-use-case-guide-dcbda7a08221>
18. Quarkus vs. Spring Boot - LogicMonitor, acceso: noviembre 9, 2025,
<https://www.logicmonitor.com/blog/quarkus-vs-spring>
 19. The Rise of New Java Frameworks in 2025: Which One Should You Learn?, acceso: noviembre 9, 2025,
<https://www.techpulseinsider.com/news/the-rise-of-new-java-frameworks-in-2025-which-one-should-you-learn/>
 20. Top 10 Microservices Frameworks in 2025 - GeeksforGeeks, acceso: noviembre 9, 2025, <https://www.geeksforgeeks.org/blogs/microservices-frameworks/>
 21. Quarkus Vs. Micronaut - Digma Continuous Feedback, acceso: noviembre 9, 2025, <https://digma.ai/quarkus-vs-micronaut/>
 22. Top 10 Java Frameworks to Watch in 2025 - Mkyong.com, acceso: noviembre 9, 2025, <https://mkyong.com/java/top-10-java-frameworks-to-watch/>
 23. When will Spring's performance be like that of Quarkus? : r/SpringBoot - Reddit, acceso: noviembre 9, 2025,
https://www.reddit.com/r/SpringBoot/comments/1lq57tq/when_will_springs_performance_be_like_that_of/
 24. Five Java Trends Shaping Enterprise Development in 2025 - Dreamix, acceso: noviembre 9, 2025,
<https://dreamix.eu/insights/enterprise-development-java-trends-2025/>
 25. GraalVM vs. Traditional JVM: Is It Time to Switch? | by Himanshu Upadhayay | The Backend Deck | Medium, acceso: noviembre 9, 2025,
<https://medium.com/the-backend-deck/graalvm-vs-traditional-jvm-is-it-time-to-switch-97973bebc4b9>
 26. Top Java Trends 2025: From Project Loom to AI Integration - Brilworks, acceso: noviembre 9, 2025, <https://www.brilworks.com/blog/java-trends/>
 27. The Ahead of Time (AOT) Compiler • 2025 - Incus Data Programming Courses, acceso: noviembre 9, 2025, <https://incusdata.com/blog/the-aot-compiler>
 28. Why GraalVM?, acceso: noviembre 9, 2025,
<https://www.graalvm.org/why-graalvm/>
 29. Native Image for Java Microservices – Faster startup times and smaller memory footprint, acceso: noviembre 9, 2025,
<https://www.javacodegeeks.com/2025/10/native-image-for-java-microservices-faster-startup-times-and-smaller-memory-footprint.html>
 30. Java 25 Startup Performance for Spring Boot, Quarkus, and Micronaut, acceso: noviembre 9, 2025,
<https://gillius.org/blog/2025/10/java-25-framework-startup.html>
 31. Spring Boot vs. Quarkus: Which Java Framework to Choose? - Mad Devs, acceso: noviembre 9, 2025, <https://maddevs.io/blog/spring-boot-vs-quarkus/>
 32. Spring Boot vs Quarkus: A Migration Story with Ugly Truths | by deToxic Dev | Stackademic, acceso: noviembre 9, 2025,
<https://blog.stackademic.com/we-switched-from-spring-boot-to-quarkus-heres>

[-the-ugly-truth-c8a91c2b8c53](#)

33. Top Java REST API Frameworks in 2025 | Zuplo Learning Center, acceso: noviembre 9, 2025,
<https://zuplo.com/learning-center/top-java-rest-api-frameworks>
34. Does GraalVM have any future in enterprise applications? : r/java - Reddit, acceso: noviembre 9, 2025,
https://www.reddit.com/r/java/comments/1cdmve6/does_graalvm_have_any_future_in_enterprise/
35. Boosting Spring Boot Performance with Project Loom's Virtual Threads | by Vinod Jagwani, acceso: noviembre 9, 2025,
<https://medium.com/@vinodjagwani/boosting-spring-boot-performance-with-project-loom-virtual-threads-9ac264e99c95>
36. Do Java 21 virtual threads address the main reason to switch to reactive single-thread frameworks? - Stack Overflow, acceso: noviembre 9, 2025,
<https://stackoverflow.com/questions/78318131/do-java-21-virtual-threads-address-the-main-reason-to-switch-to-reactive-single>
37. Reactive Java 2025: Project Loom + Virtual Threads Best Practices - Metadesign Solutions, acceso: noviembre 9, 2025,
<https://metadesignsolutions.com/reactive-java-2025-project-loom-virtual-threads-best-practices/>
38. Reactive Programming in Java: Benefits, Challenges & Best Practices - HBLAB GROUP, acceso: noviembre 9, 2025,
<https://hblabgroup.com/reactive-programming-in-java/>
39. Introduction to Reactive Programming :: Reactor Core Reference Guide, acceso: noviembre 9, 2025,
<https://projectreactor.io/docs/core/release/reference/reactiveProgramming.html>
40. Reactive Programming in Java: Project Reactor vs. RxJava - Java ..., acceso: noviembre 9, 2025,
<https://www.javacodegeeks.com/2024/12/reactive-programming-in-java-project-reactor-vs-rxjava.html>
41. Virtual Threads vs Reactive WebFlux: Which One Should You Use in 2025? | by MESfandiari, acceso: noviembre 9, 2025,
<https://medium.com/@mesfandiari77/virtual-threads-vs-reactive-webflux-which-one-should-you-use-in-2025-9720996b57e3>
42. Working with Virtual Threads in Spring | Baeldung, acceso: noviembre 9, 2025,
<https://www.baeldung.com/spring-6-virtual-threads>
43. Using Virtual Threads in Quarkus: Concurrency Simplified - Djamware, acceso: noviembre 9, 2025,
<https://www.djamware.com/post/6901c616ae60ff5276ef1695/using-virtual-threads-in-quarkus-concurrency-simplified>
44. Virtual Thread support reference - Quarkus, acceso: noviembre 9, 2025,
<https://quarkus.io/guides/virtual-threads>
45. Transitioning to virtual threads using the Micronaut loom carrier, acceso: noviembre 9, 2025,
<https://micronaut.io/2025/06/30/transitioning-to-virtual-threads-using-the-micronaut-loom-carrier/>

[naut-loom-carrier/](#)

46. Transitioning to virtual threads using the Micronaut loom carrier : r/java - Reddit, acceso: noviembre 9, 2025, https://www.reddit.com/r/java/comments/1lo8rws/transitioning_to_virtual_threads_using_the/
47. Virtual Threads in Java 24: We Ran Real-World Benchmarks—Curious What You Think, acceso: noviembre 9, 2025, https://www.reddit.com/r/java/comments/1lfa991/virtual_threads_in_java_24_we_r_an_realworld/
48. Java vs Python in Machine Learning: 2025 Guide - spec india, acceso: noviembre 9, 2025, <https://www.spec-india.com/blog/java-vs-python-for-machine-learning>
49. "Python vs. Java in 2025: Which Should I Focus On?" : r/learnpython - Reddit, acceso: noviembre 9, 2025, https://www.reddit.com/r/learnpython/comments/1hb10rn/python_vs_java_in_2025_which_should_i_focus_on/
50. Machine Learning in Java: Getting Started with DeepLearning4J, Tribuo, and Smile, acceso: noviembre 9, 2025, <https://hackernoon.com/machine-learning-in-java-getting-started-with-deeplearning4j-tribuo-and-smile>
51. TOP 12 Best Java Machine Learning Libraries 2025 - Stepmedia, acceso: noviembre 9, 2025, <https://stepmediasoftware.com/blog/best-java-machine-learning-libraries/>
52. Machine Learning in Java - Tribuo: Machine Learning in Java, acceso: noviembre 9, 2025, <https://tribuo.org/>
53. Introduction to Tribuo – A Java Machine Learning Library | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-ml-tribuo-guide>
54. Writing GPU-Ready AI Models in Pure Java with Babylon - YouTube, acceso: noviembre 9, 2025, <https://www.youtube.com/watch?v=DaMgGyfTSSw>
55. Project Babylon - OpenJDK, acceso: noviembre 9, 2025, <https://openjdk.org/projects/babylon/>
56. acceso: noviembre 9, 2025, <https://openjdk.org/projects/babylon/#:~:text=Babylon's%20primary%20goal%20is%20to.in%20Java%2C%20called%20code%20reflection.>
57. 2025 Is the Last Year of Python Dominance in AI: Java Comin' - The ..., acceso: noviembre 9, 2025, <https://thenewstack.io/2025-is-the-last-year-of-python-dominance-in-ai-java-comin/>
58. Writing GPU-Ready AI Models in Pure Java with Babylon, acceso: noviembre 9, 2025, <https://inside.java/2025/10/25/devoxxbelgium-writing-gpuready-ai-models-in-java/>
59. JEP 454: Foreign Function & Memory API - OpenJDK, acceso: noviembre 9, 2025, <https://openjdk.org/jeps/454>
60. Babylon OpenJDK: A Guide for Beginners and Comparison with TornadoVM - Juan Fumero, acceso: noviembre 9, 2025,

- <https://jifumero.github.io/posts/2025/02/07/babylon-and-tornadovm>
61. Enabling AI Development in Java With Project Valhalla | by Suren K - Medium, acceso: noviembre 9, 2025, <https://surenk.medium.com/enabling-ai-development-in-java-with-project-valhalla-b77993b7038a>
 62. Project Valhalla - OpenJDK, acceso: noviembre 9, 2025, <https://openjdk.org/projects/valhalla/>
 63. Project Valhalla: Bringing Value Types and Performance Efficiency to Java - Medium, acceso: noviembre 9, 2025, <https://medium.com/@vishalpriyadarshi/project-valhalla-bringing-value-types-and-performance-efficiency-to-java-83b85e00b791>
 64. What are Value Types from Project Valhalla? - java - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/29591897/what-are-value-types-from-project-valhalla>
 65. Project Valhalla - Java on the path to better performance - Pretius, acceso: noviembre 9, 2025, <https://pretius.com/blog/project-valhalla-java>
 66. OpenJDK Project Valhalla's head shares how they plan to enhance the Java language and JVM with value types, and more - Packt, acceso: noviembre 9, 2025, <https://www.packtpub.com/en-us/learning/how-to-tutorials/openjdk-project-valhalla-head-shares-how-they-plan-to-enhance-the-java-language-and-jvm-with-value-types-and-more>
 67. Spark vs Flink for a non data intensive team : r/dataengineering - Reddit, acceso: noviembre 9, 2025, https://www.reddit.com/r/dataengineering/comments/1kg7dix/spark_vs_flink_for_a_non_data_intensive_team/
 68. The Evolution of Java in 2025: Key Trends and Success Stories - SciForce, acceso: noviembre 9, 2025, <https://sciforce.solutions/blog/the-evolution-of-java-in-2025-key-trends-and-success-stories-z3553mfyl6uvijc1fccud8a5>
 69. A side-by-side comparison of Apache Spark and Apache Flink for ..., acceso: noviembre 9, 2025, <https://aws.amazon.com/blogs/big-data/a-side-by-side-comparison-of-apache-spark-and-apache-flink-for-common-streaming-use-cases/>
 70. Apache Flink™ vs Apache Kafka™ Streams vs Apache Spark ..., acceso: noviembre 9, 2025, <https://www.onehouse.ai/blog/apache-spark-structured-streaming-vs-apache-flink-vs-apache-kafka-streams-comparing-stream-processing-engines>
 71. Java vs Kotlin: Which One Should You Learn in 2025? | by Ramesh ..., acceso: noviembre 9, 2025, <https://medium.com/javaguides/java-vs-kotlin-which-one-should-you-learn-in-2025-73e0d18de508>
 72. Kotlin vs Java: Which is Better for Android App Development [2025] - Octal IT Solution, acceso: noviembre 9, 2025,

- <https://www.octalsoftware.com/blog/kotlin-vs-java-for-android-development>
73. Any Java devs switched to Kotlin? - Reddit, acceso: noviembre 9, 2025,
https://www.reddit.com/r/java/comments/1hys4fz/any_java_devs_switched_to_kotlin/
74. Top 10 Emerging Trends in Android Development for 2025 | by Dobri Kostadinov | ITNEXT, acceso: noviembre 9, 2025,
<https://itnext.io/top-10-emerging-trends-in-android-development-for-2025-94d3b425b09c>
75. What would you recommend for Android developers starting in 2025? : r/androiddev - Reddit, acceso: noviembre 9, 2025,
https://www.reddit.com/r/androiddev/comments/1m428hx/what_would_you_recommend_for_android_developers/
76. Comparing Java GUI frameworks: Vaadin, JavaFX, and Swing ..., acceso: noviembre 9, 2025,
<https://vaadin.com/blog/comparing-java-gui-frameworks-vaadin-javafx-and-swing>
77. Best Java GUI Frameworks 2025: A Comprehensive Guide | by Ronnie Rodriguez - Medium, acceso: noviembre 9, 2025,
<https://medium.com/javarevisited/best-java-gui-frameworks-2025-a-comprehensive-guide-f8dfef214d7e>
78. 15 Most Popular Java Frameworks for Building Web Apps in 2025 - Debut Infotech, acceso: noviembre 9, 2025,
<https://www.debutinfotech.com/blog/top-java-frameworks>
79. Top Java Frameworks in 2025 | Expert Guide - eLuminous Technologies, acceso: noviembre 9, 2025,
<https://eluminoustechnologies.com/blog/top-java-frameworks/>
80. 15 Java Frameworks In 2025 And Their Uses - Zennaxx, acceso: noviembre 9, 2025, <https://zennaxx.com/best-java-frameworks-you-should-know/>